Fundamentals in AI and ML

VIT BHOPAL

Online Voting System

Submitted by

KSHITIZ PRAKASH

VIT BHOPAL

25BCE10268

B11 +B12+B13

Title

Online Voting System is CLI based with file handling. This project represents a simple yet effective implementation of a voting system using Python. It focuses on demonstrating fundamental programming concepts while simulating a real-world voting environment in a command-line interface.

Introduction

The Online Voting System is a command-line based application developed using Python programming language. It enables users to register, log in, and cast their vote securely. The system ensures that each user can vote only once and stores all data persistently using text files. This makes the system simple, lightweight, and easy to understand.

Motivation

The motivation behind this project is to explore how digital systems can simplify traditional processes like voting. It also aims to strengthen programming skills such as file handling, conditional logic, and modular design while solving a real-world problem.

Problem Statement

Traditional voting systems are often time-consuming, prone to human error, and lack transparency. There is a need for a system that is efficient, reliable, and easy to manage. This project aims to address these issues through a digital approach.

Objective

The primary objective is to design and implement a secure voting system where users can register, log in, and vote. It also aims to provide an admin interface for managing candidates and votes effectively.

Existing method

Existing voting methods include paper-based systems and electronic voting machines. While these systems are widely used, they have limitations such as lack of flexibility, high cost, and limited accessibility.

pros and cons

The advantages of existing systems include simplicity and familiarity. However, they suffer from drawbacks like lack of transparency, difficulty in scaling, and susceptibility to errors and manipulation.

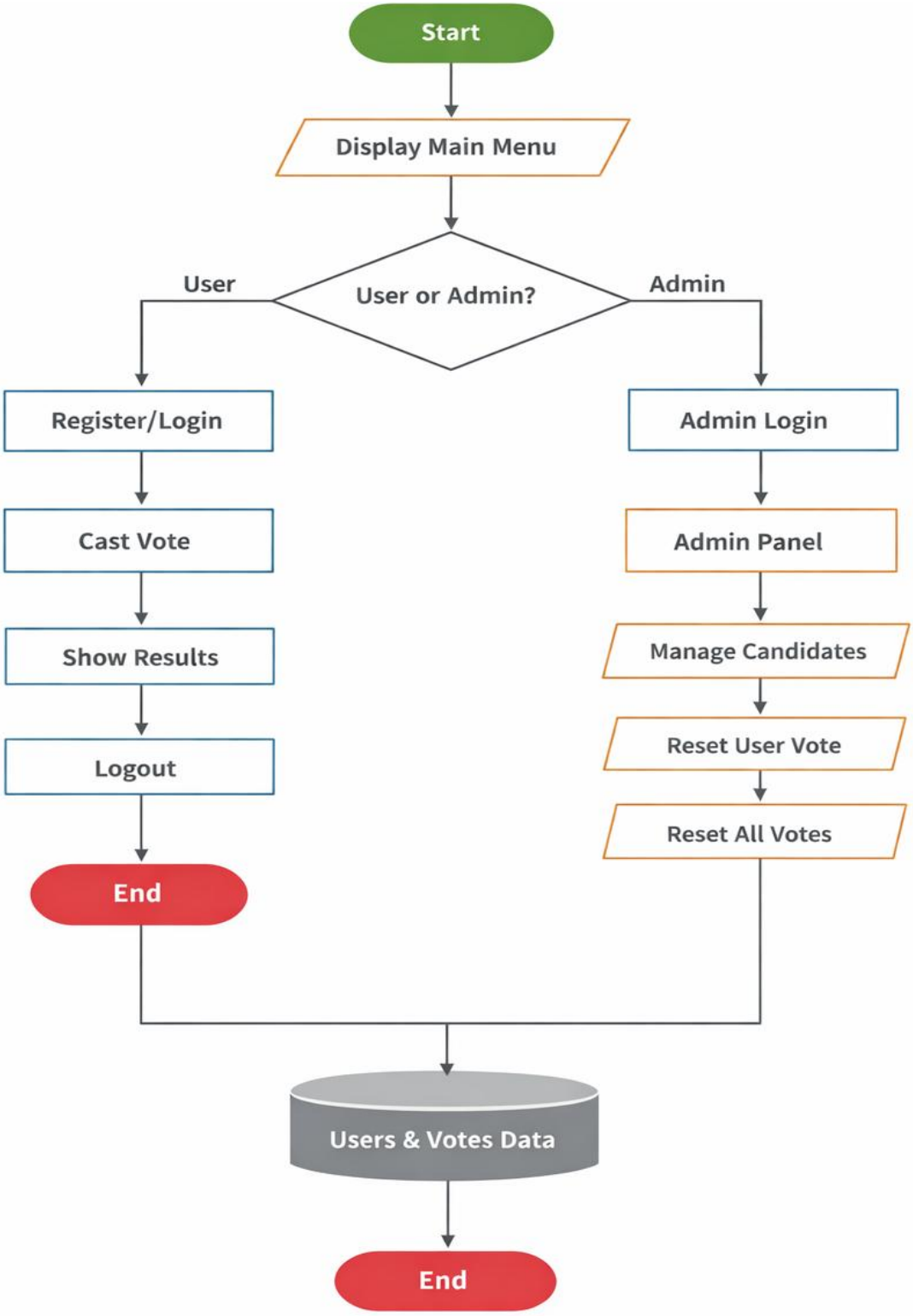
Hardware and Software

The system requires a basic computer with Python installed. Any standard operating system such as Windows, Linux, or macOS can be used. A simple text editor or IDE is sufficient for development and execution.

Methodology

The project follows a modular programming approach. Data is stored in text files, and each functionality is implemented as a separate function. The goal is to ensure simplicity, reliability, and clarity in design.

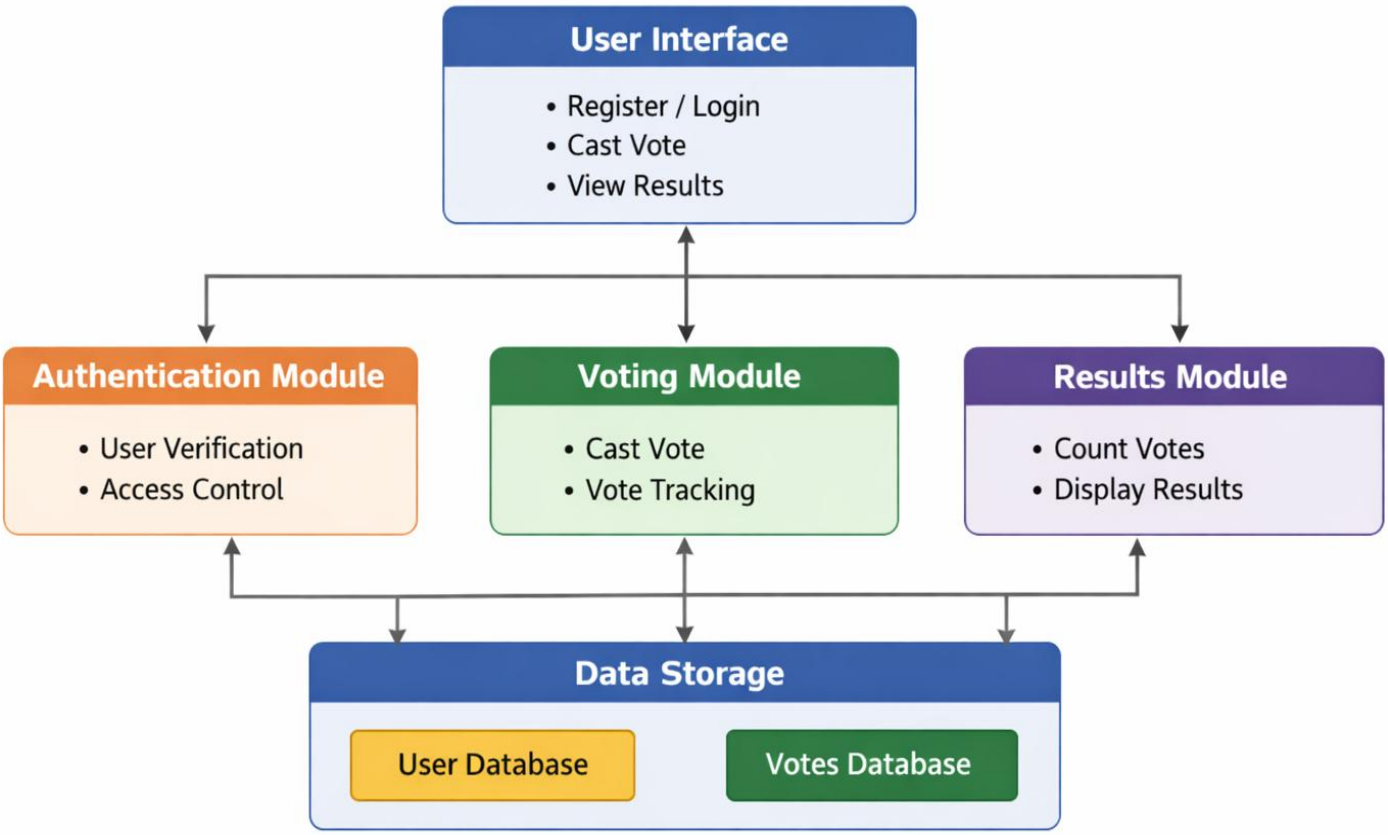
Modules Design



Algorithm

- Start
- Load users and votes from file
- Display menu
- Input choice
- If register > store new user
- If login > verify > allow voting
- If vote > check > record vote
- If show results > display votes
- If admin > add/remove > reset
- If exit > end
- Stop

Architectural Diagarm



Online Voting System Architecture

Coding

```
import os
impor time

USERS_FILE = "users.txt"
VOTES_FILE = "votes.txt"

ADMIN_USERNAME = "admin"
ADMIN_PASSWORD = "admin123"

login_attempts = {}
vote_timestamps = []

# Default candidates
candidates = {
    "BJP": 0,
    "INC": 0,
    "AAP": 0,
    "CPI": 0,
    "JMM": 0,
    "BSP": 0,
    "DMK": 0,
    "TMC": 0,
    "RJD": 0,
    "SP": 0,
    "JDS": 0,
    "SHS": 0
}
```

```
# Load users from file
def load_users():
    users = {}
    if os.path.exists(USERS_FILE):
        with open(USERS_FILE, "r")
as f:
            for line in f:
                username, password,
voted = line.strip().split(",")
                users[username] = {
                    "password":
password,
                    "voted": voted
== "True"
                }
    return users

# Save users to file
def save_users(users):
    with open(USERS_FILE, "w") as
f:
        for username, data in
users.items():

f.write(f"{username},{data['passwor
d']},{data['voted']}\n")
```

```
# Load votes from file (only for
existing candidates)
def load_votes():
    if os.path.exists(VOTES_FILE):
        with open(VOTES_FILE, "r")
as f:
            for line in f:
                name, count =
line.strip().split(",")
                if name in
candidates:   # ✅ Only update if
candidate exists

candidates[name] = int(count)

# Save votes to file (only current
candidates)
def save_votes():
    with open(VOTES_FILE, "w") as
f:
        for name, count in
candidates.items():

f.write(f"{name},{count}\n")

# Initialize data
users = load_users()
load_votes()
```

```
def register():
    global users
    username = input("Enter
username: ")
    if username in users:
        print("User already
exists!")
        return

    password = input("Enter
password: ")
    confirm_password =
input("Confirm password: ")

    if password !=
confirm_password:
        print("Passwords do not
match!\n")
        return

    users[username] = {"password":
password, "voted": False}
    save_users(users)
    print("Registration
successful!\n")
```

```
def login():
    username = input("Enter username: ")
    password = input("Enter password: ")

    if username in users and
users[username]["password"] == password:
        print("Login successful!\n")
        return username
    else:
        print("Invalid credentials!\n")
        return None

def admin_login():
    username = input("Enter admin
username: ")
    password = input("Enter admin
password: ")

    if username == ADMIN_USERNAME and
password == ADMIN_PASSWORD:
        print("Admin login
successful!\n")
        admin_panel()
    else:
        print("Invalid admin
credentials!\n")
```

```
def vote(username):
    global users
    if users[username]["voted"]:
        print("You have already
voted!\n")
        return

    print("\nCandidates:")
    for i, candidate in
enumerate(candidates.keys(), start=1):
        print(f"{i}. {candidate}")

    try:
        choice = int(input("Enter
candidate number: "))
        candidate_list =
list(candidates.keys())

        if 1 <= choice <=
len(candidate_list):
            selected =
candidate_list[choice - 1]
            candidates[selected] += 1
            users[username]["voted"] =
```

```
True

            save_votes()
            save_users(users)

            print(f"Vote casted for
{selected}!\n")
        else:
            print("Invalid choice!\n")
    except:
        print("Invalid input!\n")

def show_results():
    print("\nVoting Results:")

    # Only current candidates are used
(already ensured)
    sorted_candidates =
sorted(candidates.items(), key=lambda x:
x[1], reverse=True)

    for candidate, votes in
sorted_candidates:
        print(f"{candidate}: {votes}
votes")
```

```
# -------- ADMIN FEATURES --------

def reset_user_vote():
    global users
    username = input("Enter username to
reset vote: ")

    if username in users:
        users[username]["voted"] = False
        save_users(users)
        print("User vote reset
successfully!\n")
    else:
        print("User not found!\n")

def add_candidate():
    name = input("Enter new candidate
name: ")
    if name in candidates:
        print("Candidate already
exists!\n")
    else:
        candidates[name] = 0
        save_votes()
        print("Candidate added
successfully!\n")
```

```
def remove_candidate():
    name = input("Enter candidate name
to remove: ")
    if name in candidates:
        del candidates[name]
        save_votes()
        print("Candidate removed
successfully!\n")
    else:
        print("Candidate not found!\n")

def reset_all_votes():
    for key in candidates:
        candidates[key] = 0
    save_votes()

    for user in users:
        users[user]["voted"] = False
    save_users(users)

    print("All votes reset
successfully!\n")
```

```
def admin_panel():
    while True:
        print("\n===== Admin Panel
=====")
        print("1. Reset User Vote")
        print("2. Add Candidate")
        print("3. Remove Candidate")
        print("4. Reset All Votes")
        print("5. Back")

        choice = input("Enter choice: ")

        if choice == "1":
            reset_user_vote()
        elif choice == "2":
            add_candidate()
        elif choice == "3":
            remove_candidate()
        elif choice == "4":
            reset_all_votes()
        elif choice == "5":
            break
        else:
            print("Invalid choice!")
```

```
def main():
    while True:
        print("\n===== Online Voting
System =====")
        print("1. Register")
        print("2. Login")
        print("3. Show Results")
        print("4. Admin Login")
        print("5. Exit")

        choice = input("Enter choice:
")

        if choice == "1":
            register()

        elif choice == "2":
            user = login()
            if user:
                vote(user)

        elif choice == "3":
            show_results()

        elif choice == "4":
            admin_login()
```

```
 elif choice == "5":
            print("Exiting system...")
            break

        else:
            print("Invalid choice!")

if __name__ == "__main__":
    main()
```

Output

```
===== Online Voting System =====
1. Register
2. Login
3. Show Results
4. Admin Login
5. Exit
Enter choice: 2
Enter username: k
Enter password: k
Login successful!

You have already voted!


===== Online Voting System =====
1. Register
2. Login
3. Show Results
4. Admin Login
5. Exit
Enter choice: 5
Exiting system...
```

Key implementation

- User authentication
- one user > one vote
- AI fraud detection
- candidate management
- vote management
- admin panel
- file-based data storage
- menu driven CLI system

Outcomes

The project successfully demonstrates a working voting system with essential features. It highlights the practical application of programming concepts

Testing and refinement

The system has been tested for various scenarios including invalid inputs and duplicate users. Necessary refinements were made to improve reliability.

Real – world applications

- ✓ college and school elections
- ✓ online polls and surveys
- ✓ company voting systems
- ✓ community decision making
- ✓ event-based voting

Contribution

The project demonstrates how simple programming techniques can solve practical problems and provides a strong foundation for further development.

Limitations

The system has limitations such as lack of encryption, basic interface, basic AI fraud detection rather than full AI/ML and dependency on local storage. These can be improved in future versions.

Conclusion

The project successfully achieves its objective of creating a simple and functional voting system. It provides valuable insights into system design and implementation.

Future enhancement

Future improvements include adding a graphical user interface,ML fraud detection, integrating a database, enhancing security, and deploying the system online.

References

- Python IDE
- W3Schools
- GeeksforGeeks
- TutorialsPoint

THANKS